SatsPath — Development Status Summary

Snapshot of repo `satspath` at commit `92b8f9a` (30 Jun 2026). Plain-language summary of the main features built, their key characteristics, and their status.

Status legend: stable / done · new · experimental (flag-gated) · WIP (not merged) · needs a decision

Feature	Key characteristics	Status
Identity & signed profiles	Human alias → secp256k1 public-key identity; list of payment methods; ECDSA signature over canonical JSON; 8-char fingerprint; optional expiry	Stable
Payment methods	Lightning (Lightning Address / LNURL / BOLT12), On-chain (multiple addresses, network-aware), Ark (server + pubkey pointer)	Stable model — Lightning does a real LNURL→invoice fetch; Ark is preview-only
Routing engine	Rule-based: Lightning if amount < 100k sats → On-chain if next-block fee 20 sat/vB → Ark → else no route; live mempool fees with safe fallback	Stable (a 2nd score-based engine exists but is not wired in)
Registry & resolver chain	Local file registry, then BIP-353, then HTTP well-known, then Nostr; trust is on the signature, not the server	Stable
BIP-353 DNS payment instructions	Resolve `user@domain` to a DNSSEC-signed `bitcoin:` URI; can also publish instructions; DNSSEC mandatory, fails closed	New — resolver/preview only (never pays, signs, or broadcasts)
Universal URI & QR	`satspath:<alias>` and `satspath:v1:<payload>`; BIP-21 `bitcoin:`, `lightning:`, and Ark pointers; payloads screened for private data	Stable
Ownership proofs (verification)	Per-method proof with three trust tiers — cryptographic (key signature), domain-control (well-known / Lightning-Address), self-asserted; bound to identity + method, re-checked at resolve time	Done — richer than a minimal MVP needs
Unregistered-user invite / claim	Non-custodial by design: the sender never creates the receiver's keys; invite carries alias-hash, amount, expiry, warning	Done (invite side); claim is the receiver's own flow

Feature	Key characteristics	Status
UX quote JSON contract	`quote(recipient, amount_sats)` → one JSON the UI renders by `status` (ok / not_registered / no_route / invalid_signature); `PaymentMethod` embedded unchanged; CLI `quote --json`	Done (PR #25) — a second, richer shape now also exists (see note 1)
Mainnet preview mode (v2)	Resolves, routes, and builds a payment payload under mainnet rules; adds an execution `mode`, ownership tier, and `warnings`	New — preview only, no funds move and nothing is broadcast
P2P profile transfer	`export` a signed profile as JSON; `import` from file / stdin / HTTPS URL with signature + expiry checks (rejects tampered profiles)	New (offline transfer works); networked P2P (`holepunch` SDK) is WIP, not merged
Swap engine (Boltz) + ark-bridge	Submarine / reverse / chain swaps; Node.js Ark bridge sidecar	Experimental — testnet only, behind explicit flags
CLI	`init, register, show, encode, decode, quote, pay, invite, demo, dns, peer, preview`	Done — primary interface today (human text + ASCII QR; JSON on `quote`/`preview`)
Safety posture	Preview-only everywhere: no real send, no spend-signing, no broadcast; private-material screening; output masking by default	Enforced across the codebase

Notes

1. **Two quote response shapes now exist** — worth a decision before building the UI:
   - `quote --json` (router, PR #25): embeds the raw `PaymentMethod`; four-state enum; the shape our contract docs describe.
   - `preview` command (mainnet preview v2): a richer JSON with a **reshaped** method plus `mode`, per-recipient/per-method **ownership** tier, and a **warnings** list; all fields optional. These overlap but are not identical. The UX should target one of them (the `preview` shape is the newer, richer one).
2. **Scope.** The must-have MVP core — identity, signed profiles, signing/verification, routing, invite flow, and the quote JSON — is **complete and stable**. Several areas (ownership proofs, BIP-353 DNS, mainnet preview, P2P transfer, swaps) go **beyond a minimal MVP**: ambitious, but the UX-facing core is stable.
3. **Nothing executes real payments.** By design this is a signed-profile *resolver + router + preview*; real send/broadcast is intentionally out of scope for now.